Universidad Argentina de la Empresa

Materia: <u>PROGRAMACIÓN ORIENTADA A OBJETOS</u>



Integrantes del equipo:

- Núñez, Manuel | LU: 1138585
- Vasquez, Victoria | LU: 1218864
- Gonzalez Miel Francisco Augusto

Docente: León Maria Angela

Repositorio github: https://github.com/vicfabelo/TpProgramacionOO/invitations

Introducción

Este documento corresponde a la primera etapa del trabajo práctico. Elegimos la propuesta B, Agencia de Turismo: un sistema para la agencia Rumbo Sur que administra el catálogo de tours, la programación de salidas, las reservas con sus pagos y cancelaciones, la operación de cada salida y la liquidación de honorarios a los guías.

A lo largo del documento presentamos el análisis del enunciado con los requisitos y supuestos que tomamos, los casos de uso, el diagrama de clases inicial, dos diagramas de secuencia, la aplicación de principios SOLID y patrones GRASP y, por último, el diagrama de clases final con los cambios que surgieron de aplicar esos patrones. Los archivos de StarUML con todos los diagramas están en la carpeta docs/uml del repositorio.

1. Análisis del enunciado

1.1 Actores

Identificamos tres actores, que son los empleados de la agencia que usan el sistema:

Actor	Descripción	Casos de uso
Vendedor	Atiende a los clientes, arma las reservas y registra los pagos. Cobra comisión sobre lo vendido.	CU03, CU04, CU06, CU07, CU08, CU09
Administrador	Mantiene el catálogo, programa las salidas, liquida a los guías, emite reportes y hace el control diario de vencimientos.	CU01, CU02, CU13, CU14, CU15
Coordinador de operaciones	Asigna guía y vehículo, emite la lista de pasajeros e inicia y cierra cada salida.	CU10, CU11, CU12

El cliente, los pasajeros, los guías y los proveedores no usan el sistema directamente: son datos que el sistema administra (el cliente reserva a través del vendedor). Por eso no los consideramos actores.

1.2 Requerimientos funcionales

ID	Requerimiento
RF01	Registrar tours de aventura, culturales y de naturaleza con código, nombre, descripción, duración y cupo mínimo y máximo.
RF02	Cargar el itinerario de cada tour: actividades por día y orden, con horario, lugar y proveedor (opcional).

ID	Requerimiento
RF03	Cargar tarifas por temporada con fecha de vigencia, y servicios adicionales opcionales u obligatorios.
RF04	Programar salidas de un tour con fecha y cupo.
RF05	Registrar clientes.
RF06	Crear reservas con uno o más pasajeros, calculando el precio según el tipo de pasajero, la temporada y las promociones.
RF07	Contratar servicios adicionales por pasajero.
RF08	Registrar la seña y los pagos del saldo con distintos medios de pago y en cuotas.
RF09	Cancelar reservas aplicando la política que corresponda y registrar la devolución.
RF10	Modificar reservas: reemplazar pasajeros y registrar notas de ajuste.
RF11	Asignar guía y vehículo a cada salida.
RF12	Emitir la lista de pasajeros de una salida en un archivo de texto.
RF13	Iniciar y cerrar salidas.
RF14	Liquidar los honorarios del guía de una salida finalizada.
RF15	Emitir reportes de ocupación por salida, ventas por vendedor, ingresos por tour y cancelaciones.
RF16	Controlar vencimientos: anular reservas con saldo impago y cancelar salidas que no llegaron al cupo mínimo.
RF17	Conservar los datos entre ejecuciones (archivo).

1.3 Supuestos considerados

Del enunciado tomamos las siguientes reglas:

Tours y salidas

RN01 Hay tres categorías de tour. Los de aventura exigen una edad mínima e informan el nivel de dificultad.

RN02 Una salida puede estar programada, confirmada, completa, en curso, finalizada o cancelada.

RN03 La salida se confirma cuando alcanza el cupo mínimo. Si 7 días antes no lo alcanzó, se cancela y se devuelve el 100%.

RN04 Al llegar al cupo máximo la salida pasa a completa y no admite nuevas reservas.

RN05 Un guía no puede tener dos salidas superpuestas y solo guía las categorías para las que está habilitado.

RN06 La capacidad del vehículo no puede ser menor que el cupo de la salida.

Reservas y precios

RN07 Una reserva ocupa tantos lugares como pasajeros tenga.

RN08 Tarifa por pasajero: adulto 100%, menor de 12 años 60%, menor de 3 años 0% (pero se registra igual), jubilado 90%.

RN09 Promociones: reserva anticipada (más de 60 días antes, 15%) y grupo (6 pasajeros o más, 10%). No se acumulan, se aplica la más favorable.

RN10 Los servicios adicionales se suman al precio de cada pasajero. En los tours de aventura el seguro de viaje es obligatorio.

RN11 La reserva se confirma al pagar una seña del 30%. El saldo se paga hasta 10 días antes de la salida; si no, la reserva se anula y se libera el cupo.

RN12 Pagar con tarjeta en cuotas tiene un recargo del 7%.

Cancelaciones y operación

RN13 Si cancela el cliente: con 30 días o más se devuelve el 90%; de 29 a 15 días, el 50%; de 14 a 7 días, el 20%; con menos de 7 días no hay devolución.

RN14 Si cancela la agencia se devuelve siempre el 100%. La devolución se hace por el mismo medio con el que se pagó.

RN15 Un pasajero se puede reemplazar sin cargo hasta 3 días antes, si el nuevo cumple las condiciones del tour.

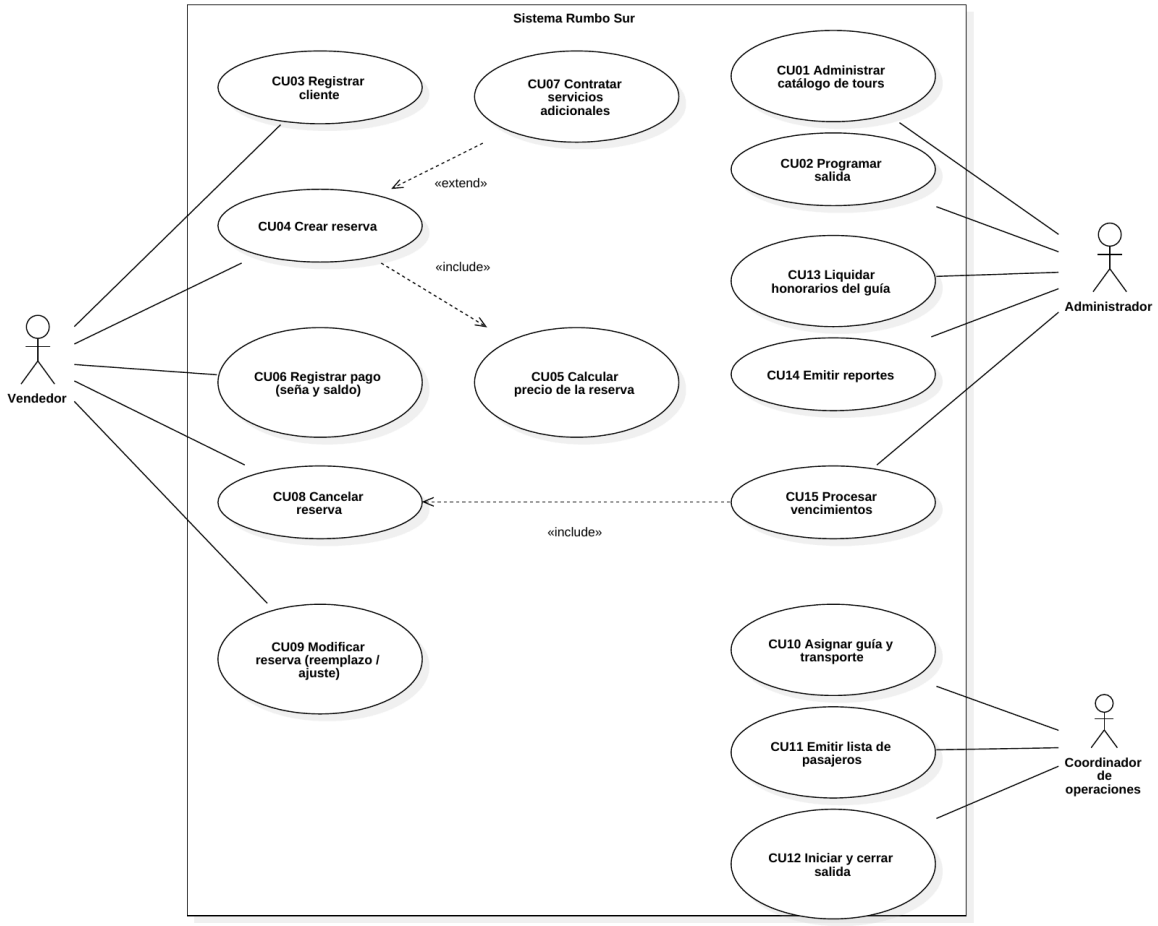
RN16 No se puede iniciar una salida sin lista de pasajeros ni sin guía asignado.

RN17 El guía cobra por cada salida finalizada un monto fijo por día más el 5% de los adicionales vendidos en esa salida.

RN18 Una reserva pagada no se elimina: las correcciones se registran como notas de ajuste.

2. Casos de uso

2.1 Diagrama de casos de uso



En el diagrama hay tres relaciones entre casos de uso. CU04 incluye a CU05 (crear una reserva siempre calcula el precio). CU07 extiende a CU04, porque contratar adicionales al reservar es opcional. CU15 incluye a CU08, ya que el control diario cancela reservas usando una política de cancelación.

2.2 Descripción de los casos de uso

Caso de uso	Actor	Precondición	Flujo principal	Flujos alternativos
CU01 Administrar catálogo	Administrador	-	Crea el tour según su categoría y le agrega actividades, tarifas y servicios.	Código repetido; actividad fuera de la duración; día y orden repetidos.
CU02 Programar salida	Administrador	El tour tiene tarifas (y seguro si es de aventura).	Elige tour, fecha y cupo; se crea la salida programada con código correlativo.	Fecha pasada; cupo fuera de los límites del tour; aventura sin seguro.
CU03 Registrar cliente	Vendedor	-	Carga nombre, apellido, documento, teléfono, correo y país.	Documento repetido; datos obligatorios vacíos.

Caso de uso	Actor	Precondición	Flujo principal	Flujos alternativos
CU06 Registrar pago	Vendedor	Reserva pendiente o confirmada.	Carga importe, medio y cuotas. Si se llega al 30% la reserva se confirma y se recalcula la salida.	Importe mayor al saldo; cuotas sin tarjeta; pago parcial después de la fecha límite.
CU07 Contratar servicios	Vendedor	Reserva modificable.	Elige pasajero y servicio del tour; se suma al total.	Servicio ya contratado o que el tour no ofrece.
CU09 Modificar reserva	Vendedor	Reserva modificable.	Reemplaza un pasajero (sin cargo) o registra una nota de ajuste.	Menos de 3 días; el nuevo pasajero no cumple el tour; ajuste sin motivo o que deja el total negativo.
CU10 Asignar guía y transporte	Coordinador	Salida en preparación.	Elige guía o vehículo y lo asigna.	Guía no habilitado o con otra salida; vehículo chico u ocupado.
CU11 Emitir lista de pasajeros	Coordinador	Salida confirmada o completa.	Genera la lista ordenada y la guarda en un archivo de texto.	Salida no confirmada.
CU12 Iniciar y cerrar salida	Coordinador	Salida confirmada o completa.	Inicia la salida; al terminar la cierra y las reservas pasan a finalizadas.	Falta guía o lista; estado incorrecto.
CU13 Liquidar honorarios	Administrador	Salida finalizada.	Calcula días por honorario más el 5% de los adicionales.	Salida no finalizada o ya liquidada.
CU14 Emitir reportes	Administrador	-	Elige el reporte, se muestra en una tabla y se puede exportar a archivo.	Sin datos: el reporte sale vacío.
CU15 Procesar vencimientos	Administrador	-	Anula reservas con saldo vencido y cancela salidas sin cupo mínimo a 7 días (incluye CU08).	No hay nada para procesar.

Resto de los casos de uso

Describimos en detalle los casos de uso CU04 y CU08, que son los que elegimos para los diagramas de secuencia, y CU05, que está incluido en CU04.

CU04 - Crear reserva	
Actor	Vendedor
Precondición	El cliente está registrado y la salida está programada o confirmada.
Flujo principal	1. El vendedor elige la salida, el titular y el vendedor, e indica la cantidad de pasajeros. 2. Carga los datos de cada pasajero. 3. El sistema valida que la salida admita reservas y que haya lugares. 4. Valida cada pasajero contra las condiciones del tour (edad mínima en aventura). 5. Crea la reserva, le agrega los pasajeros y les carga los servicios obligatorios. 6. Calcula el precio (CU05). 7. Actualiza el estado de la salida. 8. Guarda los datos y muestra el total, la promoción aplicada y la seña a pagar.
Flujos alternativos	3a. La salida está completa, en curso o cancelada: se informa y no se crea la reserva. 3b. No alcanzan los lugares: se informa cuántos quedan. 4a. Un pasajero no cumple la edad mínima: se informa y no se crea la reserva. 7a. Si se llegó al cupo máximo, la salida pasa a completa.
Postcondición	La reserva queda pendiente con un número correlativo y los lugares quedan ocupados en la salida.

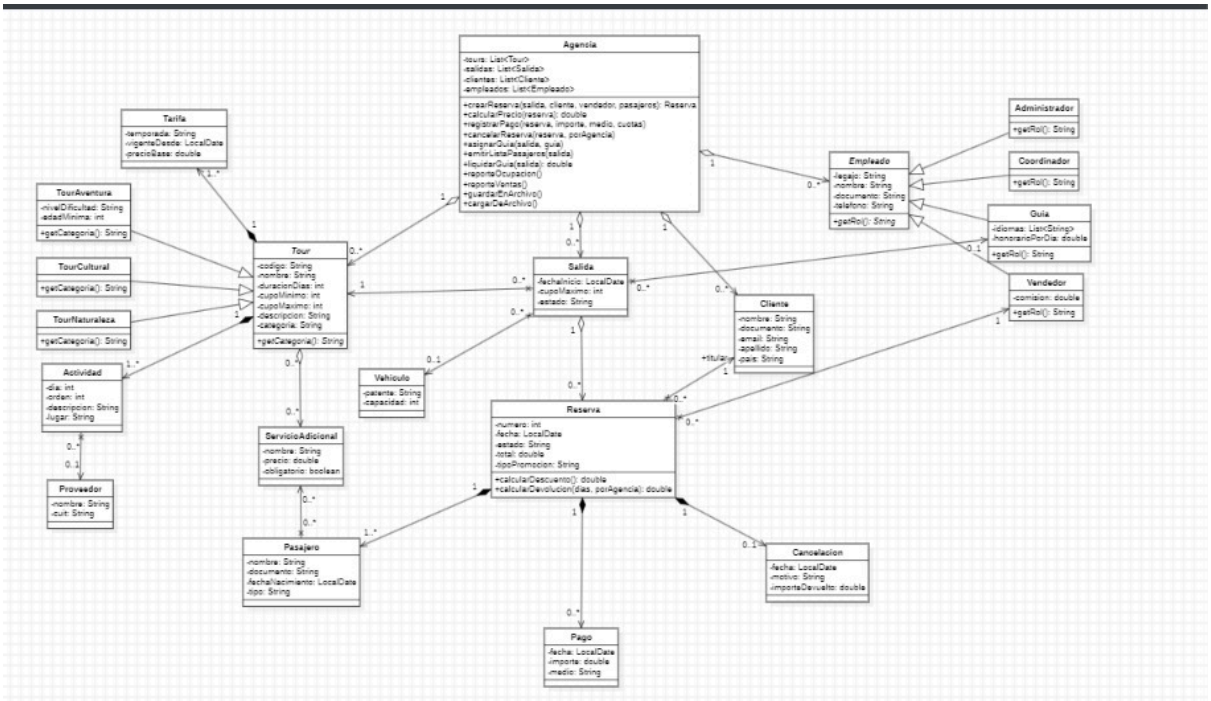
CU05 - Calcular precio de la reserva	
Actor	Incluido en CU04
Flujo principal	1. Obtiene la tarifa vigente para la temporada de la salida. 2. A cada pasajero le aplica el porcentaje según su edad y tipo. 3. Evalúa cada promoción y aplica la de mayor descuento. 4. Suma los servicios adicionales de cada pasajero.
Flujos alternativos	1a. No hay tarifa vigente: se informa y no se crea la reserva.

CU08 - Cancelar reserva	
Actor	Vendedor
Precondición	La reserva está pendiente o confirmada y la salida todavía no comenzó.
Flujo principal	1. El vendedor selecciona la reserva, indica el motivo y si la cancelación la provoca la agencia. 2. Confirma la cancelación. 3. El sistema elige la política (cliente o agencia).

	4. Calcula los días que faltan para la salida y el porcentaje a devolver. 5. Calcula la devolución por cada medio de pago. 6. Registra la cancelación, anula la reserva y libera los lugares. 7. Guarda los datos y muestra el importe devuelto.
Flujos alternativos	2a. El vendedor no confirma: no se hace nada. 3a. La reserva ya está anulada o finalizada, o la salida ya comenzó: se informa el motivo.
Postcondición	La reserva queda anulada con su cancelación registrada y el cupo se libera.

3. Diagrama de clases inicial

Es el primer modelo que armamos a partir del enunciado, antes de aplicar los patrones. Ya tiene las dos jerarquías de herencia, las dos abstractas y también aparecen las relaciones de asociación, agregación y composición.

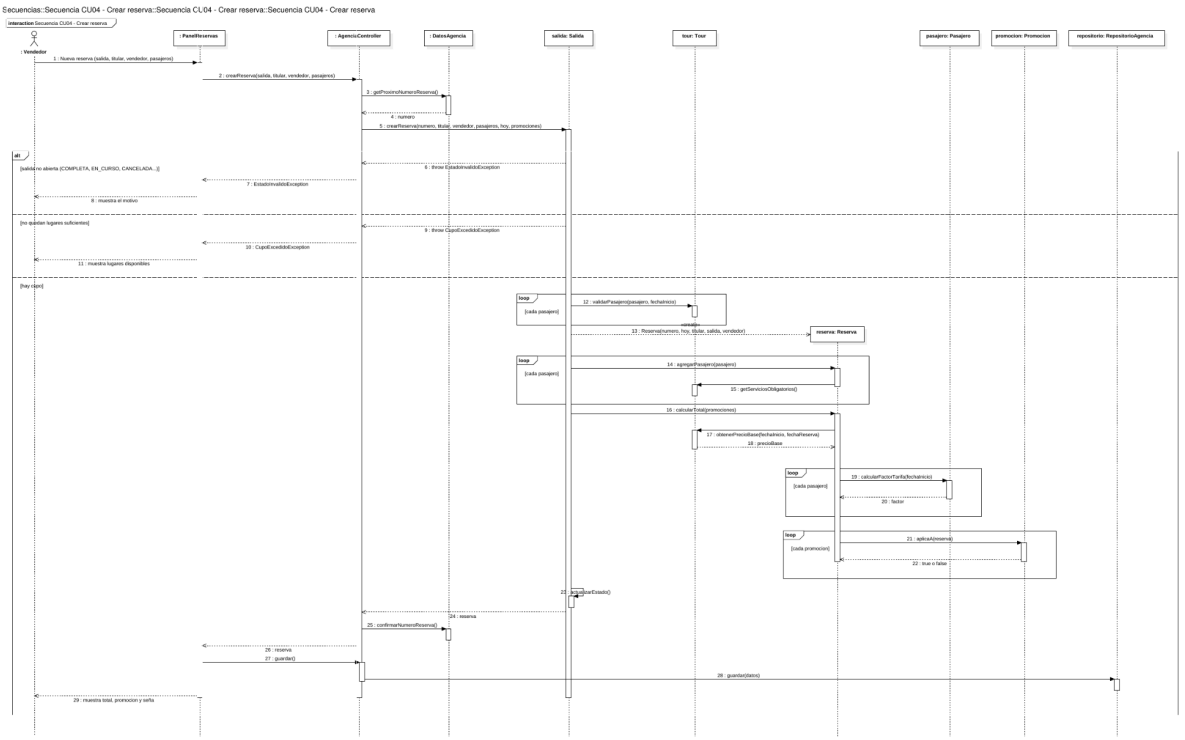


Al revisarlo encontramos varios problemas que resolvemos en la sección 5: la clase Agencia hace de todo (reservas, precios, pagos, reportes y archivos), las promociones y las cancelaciones se resuelven dentro de Reserva, y la forma de guardar los datos está atada a la clase principal.

4. Diagramas de secuencia

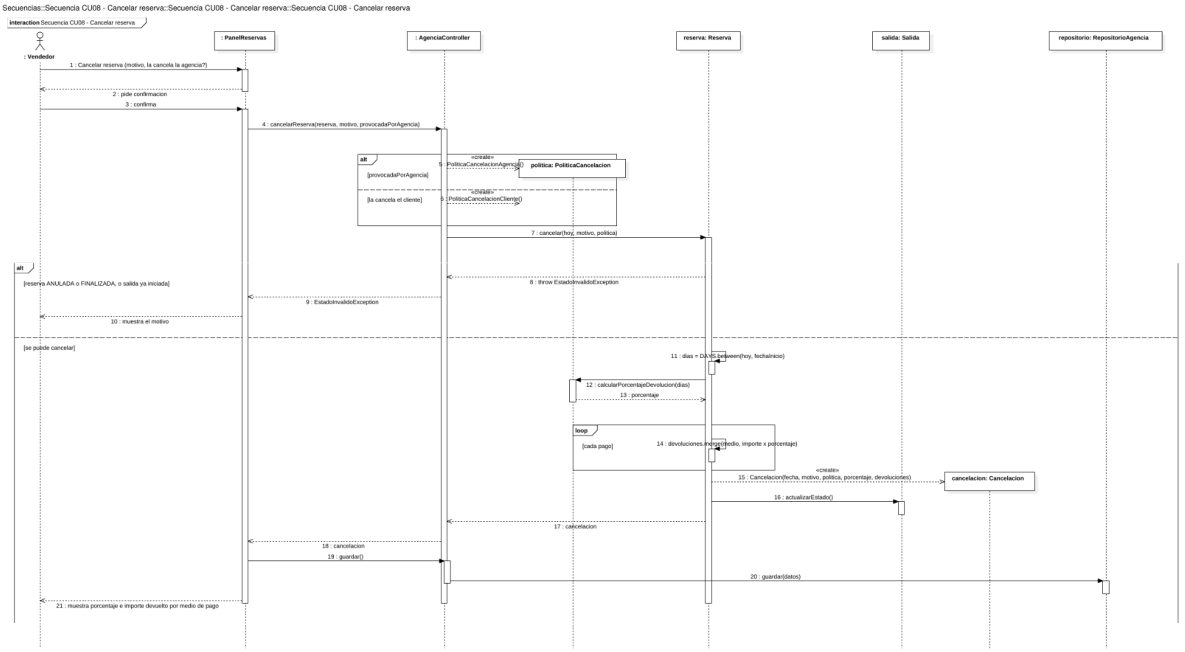
Elegimos CU04 Crear reserva, que tiene dos flujos de error (salida no disponible y cupo insuficiente), y CU08 Cancelar reserva, que tiene un flujo de error y además muestra cómo se elige la política de cancelación. Los dos usan fragmentos alt y loop, y los mensajes coinciden con los métodos del diagrama de clases final.

4.1 CU04: Crear reserva



La interfaz solo se comunica con el controlador. El controlador pide el número de reserva a DatosAgencia y delega en Salida, que valida y crea la Reserva. La reserva calcula su propio total pidiéndole la tarifa al Tour, el factor según la edad a cada Pasajero y a cada Promocion si se aplica. Si alguna validación falla se lanza una excepción propia que llega hasta la pantalla.

4.2 CU08: Cancelar reserva



El controlador crea la política según quién cancela. Reserva.cancelar() usa la política sin saber cuál es: calcula los días que faltan, pide el porcentaje, arma la devolución por medio de pago y crea la Cancelacion.

5. Aplicación de principios y patrones

Para pasar del diagrama inicial al final aplicamos tres principios SOLID y cuatro patrones GRASP. En cada caso explicamos qué problema tenía el diseño inicial, qué decidimos, en qué clases se ve y qué mejora aporta.

5.1 Principios SOLID

Principio de responsabilidad única (SRP)

En el diseño inicial la clase Agencia creaba reservas, calculaba precios, registraba pagos, armaba reportes y además leía y escribía el archivo de datos. Cualquier cambio, como agregar un reporte o cambiar el formato del archivo, obligaba a modificar la misma clase.

Por eso separamos cada responsabilidad en una clase distinta. AgenciaController solo coordina los casos de uso, GeneradorReportes arma los reportes, RepositorioArchivo guarda y lee los datos, ExportadorTexto escribe los archivos de texto y DatosAgencia solo contiene la información. En la interfaz, cada panel atiende un grupo de casos de uso.

Así las clases quedan más chicas y más fáciles de entender, y se pueden probar por separado: por ejemplo, los reportes se prueban sin la interfaz y sin archivos.

Principio abierto/cerrado (OCP)

Reserva tenía un atributo tipoPromocion y métodos con un if para cada promoción y para cada porcentaje de devolución. Agregar una promoción o una política nueva obligaba a modificar Reserva.

Definimos las interfaces Promocion y PoliticaCancelacion, con una clase por cada regla: PromocionReservaAnticipada, PromocionGrupo, PoliticaCancelacionCliente, PoliticaCancelacionAgencia y PoliticaCancelacionFaltaDePago. Los métodos Reserva.calcularTotal() y Reserva.cancelar() trabajan con la interfaz y no con cada clase.

La ventaja se vio durante el diseño: la anulación por falta de pago se agregó como una clase nueva sin tocar Reserva, y lo mismo pasaría si mañana se agrega, por ejemplo, una promoción de temporada baja.

Principio de inversión de dependencias (DIP)

En el diseño inicial la clase principal dependía directamente de cómo se guardaba el archivo, así que no se podía probar sin escribir en disco ni cambiar la forma de guardar los datos.

Ahora el controlador depende de la interfaz RepositorioAgencia y recibe la implementación por constructor; también recibe el reloj del sistema (Clock). Main crea un RepositorioArchivo y se lo pasa a AgenciaController, mientras que en las pruebas se le pasa un repositorio en memoria y un reloj con una fecha fija.

Esto permite probar los vencimientos "adelantando" la fecha y, si en el futuro se quisiera guardar en una base de datos, no habría que tocar el controlador.

5.2 Patrones GRASP

Experto en información

En el inicial, Agencia.calcularPrecio() necesitaba conocer la edad de cada pasajero, las tarifas del tour y los pagos de la reserva, es decir, dependía de datos de muchas clases.

Decidimos que cada cálculo lo haga la clase que tiene la información: Pasajero.calcularFactorTarifa() porque conoce su edad y tipo, Tour.obtenerPrecioBase() porque conoce sus tarifas, Reserva.getSaldo() y getMontoSena() porque conoce sus pagos, Salida.getLugaresDisponibles() porque conoce su cupo y sus reservas, y Guia.estaDisponible() porque conoce sus salidas.

De esta forma baja el acoplamiento, la lógica queda cerca de los datos y cada regla de negocio está en un solo lugar.

Creador

En el diagrama inicial las reservas las creaba Agencia, que no las contiene directamente. Aplicamos Creador: crea el objeto la clase que lo contiene o lo registra.

Salida.crearReserva() crea la Reserva, y el constructor de Reserva tiene visibilidad de paquete para que nadie más pueda crearla. A su vez, Reserva crea sus Pago, NotaAjuste y Cancelacion, y Salida crea su Liquidacion.

Así las validaciones de cupo y de estado se hacen siempre antes de crear la reserva y no hay forma de saltearlas.

Controlador

Si la interfaz gráfica llamara directamente a las clases del dominio, la lógica de los casos de uso quedaría mezclada con el código de los botones.

Usamos un controlador de fachada, AgenciaController, que recibe los eventos del sistema y tiene un método por caso de uso (crearReserva, registrarPago, cancelarReserva, asignarGuia, procesarVencimientos, entre otros). Los paneles de la interfaz solo leen los campos y llaman al controlador.

Gracias a esto la interfaz se podría cambiar, por ejemplo por una de consola, sin tocar el dominio, y los casos de uso se pueden probar sin pantalla.

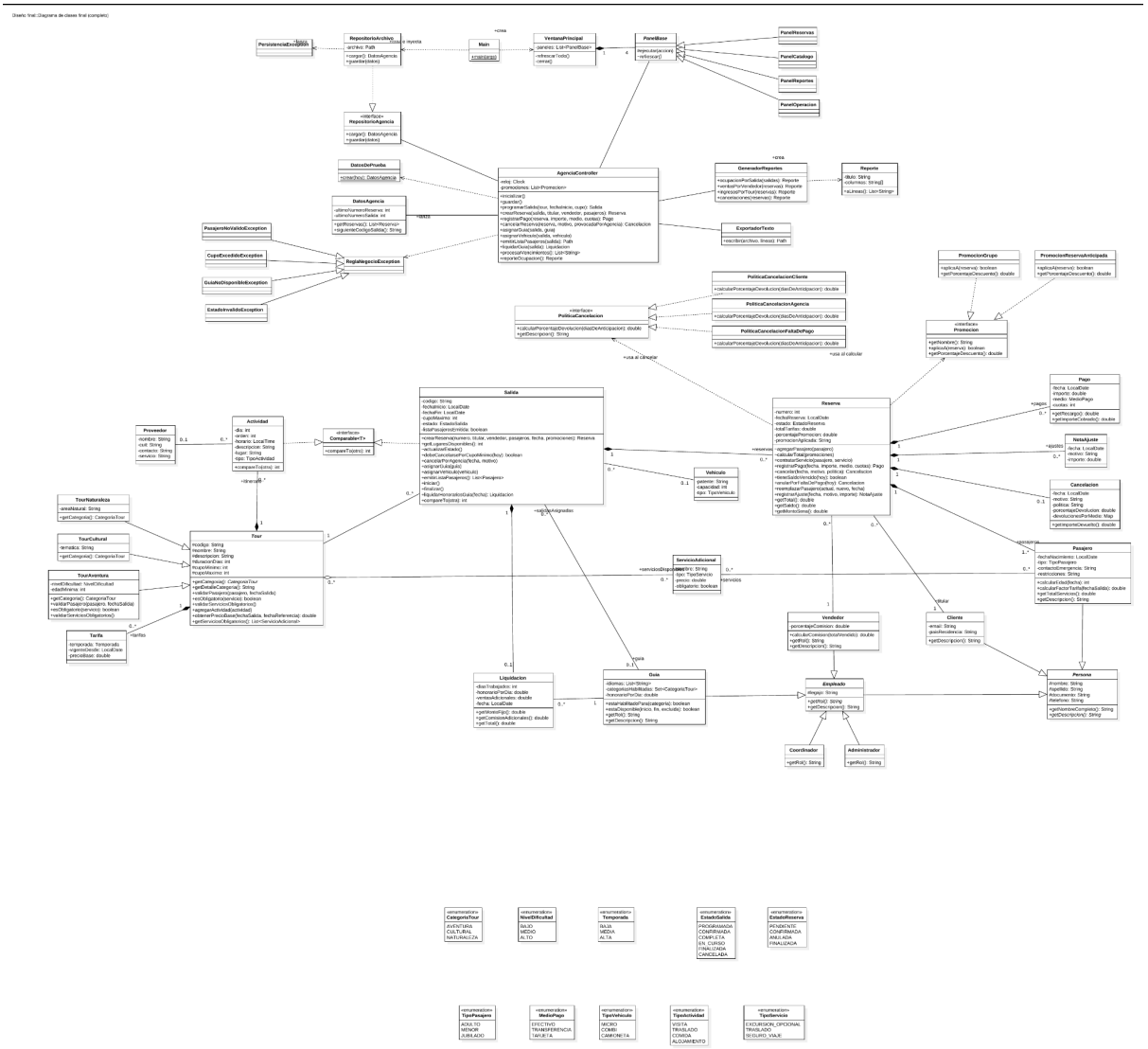
Polimorfismo

Las diferencias entre categorías de tour y entre políticas se resolvían preguntando por el tipo, con condiciones como if (categoría == AVENTURA).

Hicimos que cada tipo responda con su propio comportamiento sobrescribiendo métodos. TourAventura sobrescribe validarPasajero(), esObligatorio() y validarServiciosObligatorios(); las tres subclases de Tour sobrescriben getCategoria() y getDetalleCategoria(); Persona y Empleado definen getDescripcion() y getRol(); y cada implementación de Promocion y PoliticaCancelacion tiene su propia regla.

Con esto Salida y Reserva tratan igual a todos los tours y a todas las políticas, y agregar una categoría nueva no cambia el código que las usa.

6. Diagrama de clases final



El diagrama final incorpora los cambios de la sección anterior

Las enumeraciones no se conectan con flechas porque se usan como tipo de los atributos de otras clases (por ejemplo, estado: EstadoReserva en Reserva, medio: MedioPago en Pago, temporada: Temporada en Tarifa). Representan valores fijos y evitan usar textos que se puedan escribir mal

6.1 Cambios respecto del diagrama inicial

En la siguiente tabla resumimos los cambios entre los dos diagramas y el principio o patrón que los justifica.

Cambio	Justificación
Agencia se dividió en AgenciaController, DatosAgencia, GeneradorReportes, RepositorioArchivo y ExportadorTexto.	SRP(respuesta única) y Controlador. Agencia hacia todo, así que se dividió para que cada clase tenga una tarea y la interfaz hable solo con el controlador.
Se agregaron las interfaces Promocion y PoliticaCancelacion con sus implementaciones, y se quitó tipoPromocion de Reserva.	OCP(abierto cerrado) y Polimorfismo. Cada promoción y cada política es una clase aparte, así que se pueden agregar nuevas sin modificar Reserva.
Se agregó la interfaz RepositorioAgencia, que se le pasa al controlador por constructor.	El controlador depende de una interfaz y no del archivo, asi que se puede cambiar la forma de guardar datos sin tocarlo.
La Reserva la crea Salida.crearReserva(); Reserva crea Pago, NotaAjuste y Cancelacion; Salida crea Liquidacion.	Cada objeto lo crea la clase que lo contiene, asi una reserva solo se crea desde la salida, que antes valida el cupo.
Los cálculos pasaron a Pasajero, Tour, Reserva, Salida y Guia.	Cada calculo lo hace la clase que tiene los datos necesarios, asi cada regla esta en un solo lugar
Nueva superclase abstracta Persona (con atributos protected) para Cliente, Pasajero y Empleado.	Cliente, Pasajero y Empleado repiten los mismo datos así que para evitar atributos repetidos; suma un segundo nivel de herencia.
Los estados y tipos dejaron de ser String y pasaron a ser enumeraciones.	Con enumeraciones solo se permiten valores válidos, en cambio con un String se podía escribir mal un estado
Se agregaron NotaAjuste, Liquidacion y Proveedor como clases propias, y las excepciones de negocio.	Reglas RN17 y RN18; manejo de errores.
Actividad y Salida implementan Comparable, y Pasajero define un Comparator.	Permiten ordenar las actividades por dia, las salidas por fecha y los pasajeros por apellido, como pide el TPO

Cambio	Justificación
Se agregó la capa de interfaz gráfica (VentanaPrincipal y los paneles).	Interfaz con manejo de eventos.